Horn Formula Satisfiability Checker

Fatemeh Abshang

Spring 2024–2025

Abstract

This document presents a Python implementation of a satisfiability checker for Horn formulas in propositional logic. The system validates Horn clause structure and applies the unit propagation algorithm to determine satisfiability. Horn formulas are a restricted form of CNF formulas with at most one positive literal per clause, enabling polynomial-time satisfiability checking.

Contents

1 Introduction **3**
1.1 Horn Formulas . . . 3
1.1.1 Definition . . . 3
1.1.2 Types of Horn Clauses . . . 3
1.2 Project Features . . . 3
1.3 Logical Operators . . . 3

2 Algorithm Design **4**
2.1 Overview . . . 4
2.2 Clause Splitting . . . 4
2.3 Horn Clause Validation . . . 4
2.3.1 Antecedent Validation . . . 5
2.3.2 Consequent Validation . . . 5
2.4 Unit Propagation Algorithm . . . 5
2.4.1 Algorithm Explanation . . . 5

3 Examples **7**
3.1 Example 1: Satisfiable Formula . . . 7
3.2 Example 2: Unsatisfiable Formula . . . 7
3.3 Example 3: Complex Formula . . . 7

4 Complexity Analysis **8**
4.1 Time Complexity . . . 8
4.2 Space Complexity . . . 8
4.3 Comparison with General SAT . . . 8

5 Correctness **8**
5.1 Soundness . . . 8
5.2 Completeness . . . 8

6 Usage Instructions **9**
6.1 Input Format 9
6.2 Running the Program 9
6.3 Output Format 9

7 Applications **10**

8 Limitations and Extensions **10**
8.1 Current Limitations 10
8.2 Potential Enhancements 10

9 Conclusion **10**

A Complete Source Code **11**

1 Introduction

1.1 Horn Formulas

1.1.1 Definition

A **Horn clause** is a clause (disjunction of literals) with at most one positive literal. In implication form:

$$(p_1 \wedge p_2 \wedge \ldots \wedge p_n) \rightarrow q$$

Where $p_1, p_2, \ldots, p_n$ are positive literals (antecedents) and q is either a positive literal (consequent) or $\perp$ (false).

A **Horn formula** is a conjunction of Horn clauses:

$$\bigwedge_{i=1}^{m} (\text{Horn clause}_i)$$

1.1.2 Types of Horn Clauses

- **Definite clause**: $(p_1 \wedge \ldots \wedge p_n) \rightarrow q$ where q is a positive literal
- **Goal clause**: $(p_1 \wedge \ldots \wedge p_n) \rightarrow \perp$ (no positive consequent)
- **Fact**: $\top \rightarrow q$ (no antecedents, q is always true)

1.2 Project Features

- Parse Horn formulas in CNF with implications
- Validate Horn clause structure
- Apply unit propagation algorithm
- Determine satisfiability in polynomial time
- Output satisfying assignment when satisfiable

1.3 Logical Operators

Operator	Symbol	Usage
Conjunction	$\wedge$	Separates antecedents and clauses
Implication	$\rightarrow$	Connects antecedent to consequent
True	$\top$	Constant true value
False	$\perp$	Constant false value

Table 1: Logical operators in Horn formulas

2 Algorithm Design

2.1 Overview

The satisfiability checker follows these steps:

1. **Input**: Read and validate formula structure
2. **Parse**: Split into individual clauses
3. **Validate**: Check Horn clause format
4. **Extract**: Separate antecedents and consequents
5. **Propagate**: Apply unit propagation iteratively
6. **Output**: Report satisfiability and assignment

2.2 Clause Splitting

The formula is split by top-level $\wedge$ operators:

```python
def split_clauses(formula):
    # Remove outer parentheses if fully parenthesized
    if formula.startswith('(') and formula.endswith(')'):
        # Check if parentheses enclose entire formula
        formula = remove_outer_parens_if_needed(formula)

    # Split by top-level AND operators
    clauses = []
    current_start = 0
    depth = 0

    for i, char in enumerate(formula):
        if char == '(':
            depth += 1
        elif char == ')':
            depth -= 1
        elif char == AND and depth == 0:
            clause = formula[current_start:i].strip()
            clauses.append(clean_clause(clause))
            current_start = i + 1

    # Add final clause
    clauses.append(clean_clause(formula[current_start:]))
    return clauses
```

Listing 1: Clause splitting function

2.3 Horn Clause Validation

Each clause must have the form: $(p_1 \wedge \ldots \wedge p_n) \rightarrow q$

Algorithm 1 Horn Clause Validation

```
function IsValidClause(clause)
    if →∉ clause then
        return false
    end if
    parts ← split(clause, →)
    if length(parts) ≠ 2 then
        return false
    end if
    antecedent ← parts[0]
    consequent ← parts[1]
    return IsValidAntecedent(antecedent) and IsValidConsequent(consequent)
end function
```

2.3.1 Antecedent Validation

Valid antecedents are conjunctions of:

- Positive literals: lowercase letters
- Constants: $\top$ or $\bot$

2.3.2 Consequent Validation

Valid consequents are:

- Single positive literal: lowercase letter
- Constants: $\top$ or $\bot$

2.4 Unit Propagation Algorithm

The core satisfiability checking algorithm:

2.4.1 Algorithm Explanation

1. **Initialize**: All variables start as false
2. **Iterate**: Check each clause
    - If all antecedents are true, the consequent must be true
    - If consequent is $\bot$, formula is unsatisfiable
    - Otherwise, set consequent to true and continue
3. **Fixed Point**: Repeat until no changes occur
4. **Result**: If no contradiction found, formula is satisfiable

Algorithm 2 Horn Satisfiability via Unit Propagation

```
function HornSatisfiable(clauses)
    variables ← ∅
    hornClauses ← []
    for each clause in clauses do
        (antecedents, consequent) ← Extract(clause)
        Add (antecedents, consequent) to hornClauses
        Add all variables from antecedents and consequent to variables
    end for
    assignment ← {v ↦ false | v ∈ variables}
    changed ← true
    while changed do
        changed ← false
        for each (antecedents, consequent) in hornClauses do
            if All antecedents are true then
                if consequent = ⊥ then
                    return Unsatisfiable
                end if
                if consequent is not yet true then
                    assignment[consequent] ← true
                    changed ← true
                end if
            end if
        end for
    end while
    return Satisfiable, assignment
end function
```

3 Examples

3.1 Example 1: Satisfiable Formula

Input:

$$(p \to q) \land (q \to r) \land (\top \to p)$$

Execution:

1. Initial: $p = \textbf{false}, q = \textbf{false}, r = \textbf{false}$
2. Clause 3: $\top \to p$ fires, set $p = \textbf{true}$
3. Clause 1: $p \to q$ fires, set $q = \textbf{true}$
4. Clause 2: $q \to r$ fires, set $r = \textbf{true}$
5. No more changes, fixed point reached

Output:

```
Satisfiable
p
q
r
```

3.2 Example 2: Unsatisfiable Formula

Input:

$$(p \to q) \land (q \to \bot) \land (\top \to p)$$

Execution:

1. Initial: $p = \textbf{false}, q = \textbf{false}$
2. Clause 3: $\top \to p$ fires, set $p = \textbf{true}$
3. Clause 1: $p \to q$ fires, set $q = \textbf{true}$
4. Clause 2: $q \to \bot$ fires, contradiction detected!

Output:

```
Unsatisfiable
```

3.3 Example 3: Complex Formula

Input:

$$(p \land q \to r) \land (r \to s) \land (\top \to p) \land (\top \to q)$$

Output:

```
Satisfiable
p
q
r
s
```

4 Complexity Analysis

4.1 Time Complexity

- **Parsing**: $O(n)$ where n is formula length
- **Validation**: $O(m \cdot k)$ where m is number of clauses and k is average clause size
- **Unit Propagation**: $O(m \cdot v)$ where v is number of variables
  - Each iteration: $O(m \cdot k)$ to check all clauses
  - Maximum iterations: $O(v)$ (each variable set at most once)
  - Total: $O(m \cdot k \cdot v)$

Overall: $O(m \cdot k \cdot v) = O(n \cdot v)$ which is **polynomial time**

4.2 Space Complexity

- **Clause storage**: $O(m \cdot k) = O(n)$
- **Variable assignment**: $O(v)$
- **Total**: $O(n)$

4.3 Comparison with General SAT

Problem	**Time Complexity**	**Class**
General SAT	$O(2^n)$	NP-complete
Horn SAT	$O(n \cdot v)$	P (polynomial)

Table 2: Complexity comparison

The restriction to Horn clauses enables polynomial-time satisfiability checking, unlike general CNF formulas.

5 Correctness

5.1 Soundness

[Soundness] If the algorithm reports "Satisfiable", then the formula is satisfiable.

The algorithm constructs an assignment by unit propagation. Each variable is set to true only when required by a clause whose antecedents are all true. This assignment satisfies all clauses by construction.

5.2 Completeness

[Completeness] If the formula is satisfiable, the algorithm reports "Satisfiable".

Unit propagation finds the minimal satisfying assignment. For Horn formulas, if any satisfying assignment exists, the minimal one (fewest true variables) can be found by unit propagation. If the algorithm reaches a fixed point without contradiction, this minimal assignment satisfies the formula.

6 Usage Instructions

6.1 Input Format

- Variables: lowercase letters (`p, q, r, ...`)
- Conjunction: $\wedge$ (between antecedents and clauses)
- Implication: $\rightarrow$ (antecedent to consequent)
- Constants: $\top$ (true), $\perp$ (false)
- Parentheses for grouping

Example file content:

```
(p -> q) ^ (q -> r) ^ (T -> p)
```

6.2 Running the Program

```python
# Run the program
python horn_sat.py

# Enter filename
Please enter a file name:
horn_formula.txt

# Output
Satisfiable
p
q
r
```

Listing 2: Execution example

6.3 Output Format

If Satisfiable:

```
Satisfiable
[list of true variables in alphabetical order]
```

If Unsatisfiable:

```
Unsatisfiable
```

If Invalid Formula:

```
Invalid Horn Formula
```

7 Applications

Horn formulas are widely used in:

- **Logic Programming**: Prolog uses Horn clauses
- **Datalog**: Database query language based on Horn logic
- **Expert Systems**: Rule-based reasoning
- **Type Inference**: Type systems use Horn clause resolution
- **Static Analysis**: Program verification and bug detection

8 Limitations and Extensions

8.1 Current Limitations

1. Only handles definite Horn clauses (positive consequents)
2. No support for disjunction in consequents
3. No clause learning or optimization
4. Limited error reporting

8.2 Potential Enhancements

1. **Extended Horn**: Support more general Horn clauses
2. **Minimal Model**: Find smallest satisfying assignment
3. **Explanation**: Trace which clauses led to assignments
4. **DIMACS format**: Support standard input/output formats
5. **Incremental solving**: Add/remove clauses dynamically

9 Conclusion

This Horn formula satisfiability checker implements an efficient polynomial-time algorithm for a restricted but important class of propositional formulas. The unit propagation algorithm leverages the special structure of Horn clauses to achieve tractable satisfiability checking, unlike the NP-complete general SAT problem.

The implementation correctly validates Horn clause structure and applies the classical unit propagation algorithm, making it suitable for applications in logic programming, expert systems, and automated reasoning.

A Complete Source Code

```python
# Define logical symbols
NOT = '¬'
AND = '∧'
OR = '∨'
IMP = '→'
TRUE = '⊤'
FALSE = '⊥'

def get_input():
    print("Please enter a file name:")
    file_name = input()
    with open(file_name, 'r', encoding='utf-8') as file:
        formula = file.read().strip()
        formula = formula.replace(" ", "")
        return formula

def check_parentheses(formula):
    stack = []
    for char in formula:
        if char == '(':
            stack.append(char)
        elif char == ')':
            if not stack:
                return False
            stack.pop()
    return not stack

def split_clauses(formula):
    # Remove outer parentheses if fully parenthesized
    if formula.startswith('(') and formula.endswith(')'):
        balance = 0
        is_fully_parenthesized = True
        for i, char in enumerate(formula[1:-1]):
            if char == '(':
                balance += 1
            elif char == ')':
                balance -= 1
            if balance < 0:
                is_fully_parenthesized = False
                break
        if is_fully_parenthesized and balance == 0:
            formula = formula[1:-1]

    # Split by top-level AND
    clauses = []
    current_start = 0
    depth = 0

    for i, char in enumerate(formula):
```

```
        if char == '(':
            depth += 1
        elif char == ')':
            depth -= 1
        elif char == AND and depth == 0:
            clause = formula[current_start:i].strip()
            if clause.startswith('(') and clause.endswith(')'):
                clause = clause[1:-1]
            if clause:
                clauses.append(clause)
            current_start = i + 1

    final_clause = formula[current_start:].strip()
    if final_clause.startswith('(') and final_clause.endswith(')'):
        final_clause = final_clause[1:-1]
    if final_clause:
        clauses.append(final_clause)

    return clauses

def is_valid_antecedent(antecedent):
    if not antecedent:
        return False
    parts = antecedent.split(AND)
    for part in parts:
        part = part.strip()
        if part == TRUE or part == FALSE:
            continue
        if not (len(part)==1 and part.isalpha() and part.islower()):
            return False
    return True

def is_valid_consequent(consequent):
    if not consequent:
        return False
    if (len(consequent)==1 and consequent.isalpha()
        and consequent.islower()) or consequent in {TRUE, FALSE}:
        return True
    return False

def is_valid_clause(clause):
    if IMP in clause:
        parts = clause.split(IMP)
        if len(parts) != 2:
            return False
        ant = parts[0].strip()
        con = parts[1].strip()
        return is_valid_antecedent(ant) and is_valid_consequent(con)
```

```python
    return False

def extract(clause):
    parts = clause.split(IMP)
    antecedents = parts[0].strip()
    con = parts[1].strip()
    ant = [p.strip() for p in antecedents.split(AND)]
    return ant, con

def Horn_satisfiable(clauses):
    horn_clauses = []
    variables = set()

    for clause_str in clauses:
        if clause_str.startswith('(') and clause_str.endswith(')')
            :
            clause_str = clause_str[1:-1]

        if not is_valid_clause(clause_str):
            print("Invalid Horn Formula")
            return

        antecedents, consequent = extract(clause_str)
        horn_clauses.append((antecedents, consequent))

        for a in antecedents:
            if a not in {TRUE, FALSE}:
                variables.add(a)
        if consequent not in {TRUE, FALSE}:
            variables.add(consequent)

    assignment = {var: False for var in variables}

    changed = True
    while changed:
        changed = False
        for antecedents, consequent in horn_clauses:
            antecedent_is_true = True
            for a in antecedents:
                if a == FALSE:
                    antecedent_is_true = False
                    break
                elif a == TRUE:
                    continue
                elif not assignment.get(a, False):
                    antecedent_is_true = False
                    break

            if antecedent_is_true:
                if consequent == FALSE:
                    print("Unsatisfiable")
```

```
                    return
                if consequent == TRUE:
                    continue
                if not assignment.get(consequent, False):
                    assignment[consequent] = True
                    changed = True

    print("Satisfiable")
    true_vars = sorted([v for v, val in assignment.items() if val
        ])
    for var in true_vars:
        print(var)

def main():
    formula = get_input()
    if not check_parentheses(formula):
        print("Invalid Horn Formula")
        return
    clauses = split_clauses(formula)
    Horn_satisfiable(clauses)

if __name__ == "__main__":
    main()
```

Listing 3: Horn SAT Checker Implementation